



EPFL Department of Mathematics

Course Project: MPI and CUDA Parallelization of a Shallow Water Equation (SWE) Solver

MATH-454: Parallel and High-Performance Computing

Author: Selim SHERIF

June 21, 2025

Contents

1	Code Profiling	2
2	Computational Complexity	2
3	MPI Parallelization	3
3.1	Code Architecture	3
3.2	Strong Scaling Results	3
3.3	Weak Scaling Results	4
4	CUDA Parallelization	5
4.1	Code Architecture	5
4.2	Results and Discussion	5

Brief Introduction: This course project, completed as part of EPFL's **MATH-454: Parallel and High Performance Computing**, focuses on the parallelization of a Shallow Water Equation (SWE) solver. The work begins with a CPU-based implementation using MPI, and is subsequently extended to a GPU-based version leveraging CUDA. The objective is to apply the techniques learned throughout the course to accelerate a real-world numerical solver through high-performance parallel computing.

1 Code Profiling

A crucial first step in optimizing any scientific code is performance profiling, which identifies where execution time is predominantly spent. This diagnostic insight guides the prioritization of routines for parallelization.

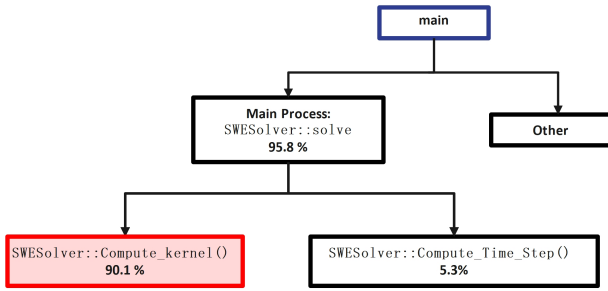


Figure 1 | Call graph profiling breakdown of the SWE solver.

Profiling reveals the following key insights:

- `Compute_kernel()` dominates with **90.1%** of the total runtime. It is therefore the primary target for GPU acceleration.
- `Compute_Time_Step()` contributes **5.3%**, This is where the CFL condition is setup.
- Remaining routines contribute marginally, with no individual function exceeding 5%.
- The overall parallelization coefficient is approximately $P \approx 0.96$, measured over the entire `solve()` routine, indicating strong scalability potential.

Unsurprisingly, the compute kernel accounts for the vast majority of the runtime, with additional cost concentrated in the CFL stability enforcement. This confirms that the code is compute-bound and would greatly benefit from parallelization strategies. These insights guide the subsequent application of MPI and CUDA to accelerate the solver.

2 Computational Complexity

The per-time-step computational complexity of the parallel Shallow Water Equations solver is analyzed for a total grid size of $N = n^2$, evenly distributed across ν processes. The `compute_kernel` routine updates the conservative variables (h, hu, hv) using a fixed stencil derived from first-order finite difference schemes. This leads to a local computational cost of $\mathcal{O}(N/\nu)$ per process. Communication arises from two main sources:

- **Ghost cell exchange:** Each process communicates with its neighboring subdomains to update halo layers. Since each process holds a block of rows, the amount of exchanged data scales with the subdomain perimeter, yielding a communication cost of $\mathcal{O}(\sqrt{N})$.
- **CFL condition enforcement:** Stability requires computing the global maximum wave speed at every time step. This is done via an `MPI_Allreduce` operation, introducing a synchronization cost of $\mathcal{O}(\log \nu)$.

The total parallel complexity per time step can thus be expressed as:

$$T_{\text{parallel}} = \mathcal{O}\left(\frac{N}{\nu} + \sqrt{N} + \log \nu\right) \quad (1)$$

This expression highlights the trade-off between computation and communication. As ν increases, the workload per process decreases, but overhead from halo exchange and global reductions grows. The term $\mathcal{O}(\sqrt{N} + \log \nu)$ captures the main scalability bottlenecks at high core counts.

3 MPI Parallelization

3.1 Code Architecture

The MPI parallelization of the SWE solver employs a row-wise domain decomposition strategy. Initial conditions are fully prepared on the root process (rank 0), either from analytical expressions or from HDF5 input files. The constructor on the root node handles the entire setup: initializing physical variables, computing derivatives, and preparing data structures. Once the setup is complete, the domain data is scattered to all ranks, with each MPI process receiving its assigned block of rows. Each process also maintains ghost layers at the top and bottom boundaries to facilitate halo exchange with neighboring ranks. The main computational loops are confined to each process’s local subdomain, enabling independent and efficient updates.

A critical global operation is the enforcement of the CFL condition via `MPI_Allreduce`, which ensures all processes synchronize their time steps based on the minimum stable value across the domain.

While a two-dimensional domain decomposition was considered, the decision to adopt a one-dimensional row-wise strategy was motivated by its conceptual simplicity and sufficient performance for the problem at hand. Nevertheless, extending the solver to support 2D partitioning remains a promising direction for improving load balancing and scalability in more demanding scenarios.

3.2 Strong Scaling Results

A strong scaling analysis is performed by varying the number of cores for a fixed problem size. Ideally, runtime decreases linearly with the number of processes, yielding speedup $S = \nu$ and efficiency $E = 1$. In practice, performance is bounded by Amdahl’s Law:

$$S_{\text{Amdahl}}(\nu) = \frac{1}{(1 - P) + \frac{P}{\nu}}, \quad P = 0.96$$

Figures 2 and 3 summarize the strong scaling behavior across all test cases. The results reflect consistent performance trends that depend more on grid resolution and parallel overhead than on the specific initial condition.

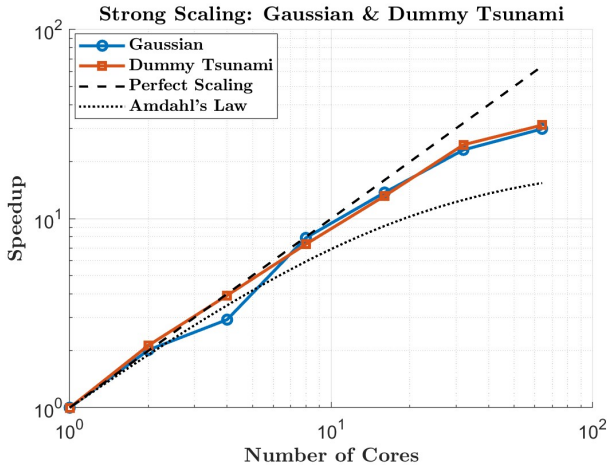


Figure 2 | Strong scaling for the Gaussian and Dummy Tsunami cases on a 1001×1001 grid. Both show nearly identical performance, confirming that stencil computations dominate regardless of the initial condition.

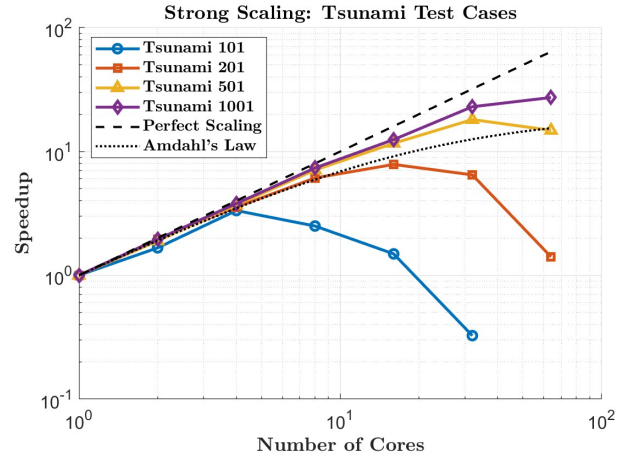


Figure 3 | Strong scaling for the realistic Tsunami case at four resolutions. Higher grid sizes scale well across cores, while smaller grids suffer early from communication overhead and workload imbalance.

Key observations:

- Strong scaling is excellent across all tests, with larger grids, particularly the 1001×1001 cases, occasionally outperforming Amdahl's prediction. This suggests that the assumed $P = 0.96$ may underestimate the solver's effective parallel fraction.
- The Gaussian and Dummy Tsunami cases exhibit nearly identical speedup curves, confirming that performance is driven by the underlying stencil computation and grid size, rather than scenario-specific initial conditions.
- Scaling breaks down earlier for low-resolution grids, as smaller per-core workloads amplify communication and synchronization costs. This effect is clearly visible in Figure 3 for the 101 and 201 grids.

3.3 Weak Scaling Results

Weak scaling performance was evaluated by proportionally increasing the total problem size with the number of processes, keeping the per-process workload constant, i.e., $N/\nu = \text{const}$. In this setting, computation per process remains fixed, while communication costs grow due to the increasing perimeter-to-area ratio of subdomains. Theoretically, communication overhead scales with the subdomain boundary length, i.e., $\mathcal{O}(\sqrt{N})$. Since $N \propto \nu$, this results in a per-process communication cost of $\mathcal{O}(\sqrt{\nu})$, while computation stays constant. Consequently, weak scaling efficiency is expected to follow:

$$E_{\text{weak}}(\nu) \propto \frac{1}{1 + c_1\sqrt{\nu} + c_2 \log \nu} \quad (2)$$

where c_1 and c_2 represent communication and synchronization costs, respectively. In the communication-dominated regime, this simplifies to:

$$E_{\text{weak}}(\nu) \sim \mathcal{O}\left(\frac{1}{\sqrt{\nu}}\right) \quad (3)$$

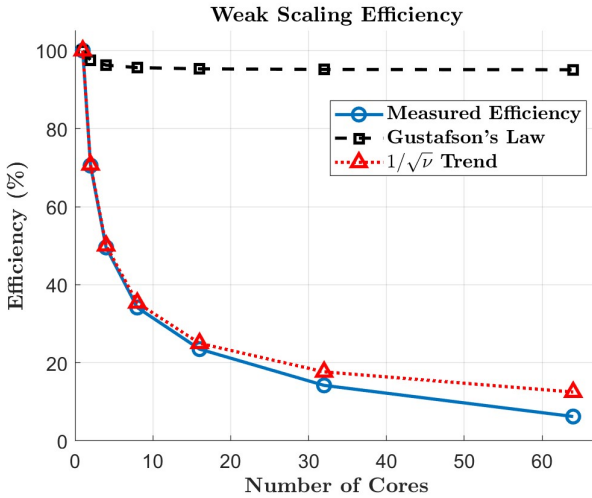


Figure 4 | Weak scaling efficiency as a function of core count. Measured efficiency is compared with Gustafson's Law and a $1/\sqrt{\nu}$ trend representing communication-limited behavior. The tests use a Gaussian initial condition, starting from a 1001×1001 grid and scaling the domain proportionally with the number of processes.

The results of the weak scaling test are presented in Figure 4, and the following takeaways can be extracted:

Key observations:

- The measured efficiency consistently diverges from Gustafson's Law across all tested core counts, suggesting that synchronization and halo exchanges are persistent performance bottlenecks throughout the scaling regime.
- Efficiency steadily declines with increasing core count, primarily due to rising communication overhead relative to fixed per-core computation.
- The theoretical $1/\sqrt{\nu}$ reference curve captures the degradation trend remarkably well, validating the communication-scaling model. A minor deviation is visible, but the overall agreement is strong, supporting its predictive accuracy (possibly due to the $\log \nu$ synchronization term discussed above).

4 CUDA Parallelization

4.1 Code Architecture

The CUDA version of the SWE solver parallelizes the core update kernel by assigning one GPU thread per grid cell. The 2D domain is mapped onto a thread block layout with launch parameters `tpb_x` and `tpb_y`, configured at runtime in the constructor, which also handles memory allocation and data transfer.

Fields (h, hu, hv) and topography gradients reside in global memory and are updated entirely on the device, including boundary conditions. The adaptive time step is computed via a reduction kernel, with only the final value transferred to the host. To reduce overhead, inputs are copied to the GPU at initialization, and only selected outputs are retrieved during execution. The solver uses double-buffered arrays for in-place updates and efficient time stepping on the device.

4.2 Results and Discussion

A sweep over the number of CUDA threads per block (TPB) was conducted to evaluate kernel performance. The average time per iteration was measured across three test cases on a 1001×1001 grid: Gaussian initialization, Dummy Tsunami, and the 1001 Tsunami. TPB values ranged from 1 to 1024 (powers of two), using square blocks (`tpb_x = tpb_y`), effectively altering the number of launched blocks and revealing key performance trends. The upper limit of 1024 threads reflects the architectural constraint on maximum threads per block for standard GPU kernels.

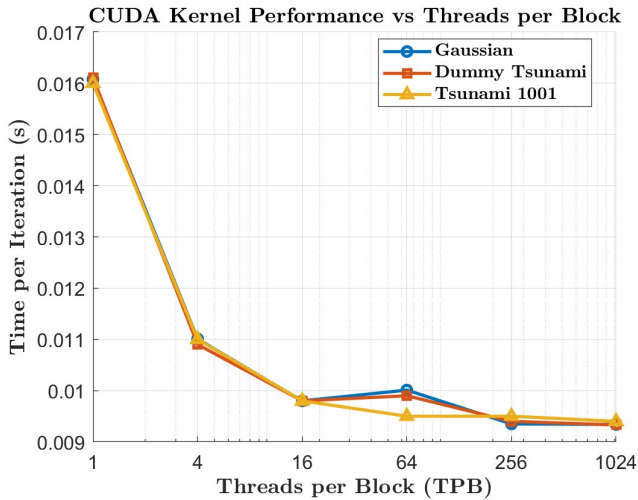


Figure 5 | Average iteration time vs. Threads per Block (TPB) for three test cases. The x-axis is log-scaled to highlight performance trends.

Two main performance regimes are observed:

1. **Low TPB (1–16):** Poor performance due to excessive kernel launch overhead and underutilized GPU cores.
2. **Moderate to High TPB (16–1024):** Optimal performance is achieved in this range, where improved occupancy and reduced scheduling overhead lead to faster iteration times.

Discussion: Although a U-shaped performance curve might be expected, no significant rise in iteration time is observed at high TPB values. Instead, performance plateaus beyond $\text{TPB} = 16$. This is likely due to the large number of available blocks, which ensures sufficient occupancy of the GPU’s Streaming Multiprocessors (SMs). On the NVIDIA V100 GPU used (with 80 SMs), performance typically degrades when the block count becomes too low, yet this effect was not clearly observed, even in smaller matrices, suggesting that kernel launch overhead or divergence was not a limiting factor in this setting.

Concluding Remarks Profiling revealed a parallel efficiency of $P \approx 0.95$, confirming the solver’s scalability and guiding optimization efforts. This project demonstrated the effective use of hybrid parallelism to accelerate a Shallow Water Equation solver. The compute kernel was identified as the dominant cost, making the code well-suited for MPI and CUDA acceleration. MPI achieved strong scaling on large grids and exhibited expected weak scaling limits due to communication overheads scaling with $\sqrt{\nu}$ and $\log \nu$. CUDA provided further speedups, with optimal performance for $\text{TPB} > 16$, and no observed slowdown observed at higher values. These results highlight the potential of parallelization and lay the groundwork for future extensions, including hybrid MPI-CUDA schemes, OpenMP integration.